Practical 8

Part A:

Aim:

1. Guarded calculator: read two numbers and an operator; throw a custom DivideByZeroException and catch invalid-number input separately, printing a clear message each time; loop until a valid calculation succeeds and use finally to log each attempt.

Code:

GuardedCalculator.java

```
import java.util.Scanner;
import java.util.InputMismatchException;

// Custom exception
class DivideByZeroException extends Exception {

    public DivideByZeroException(String message) {
        super(message);
    }
}

public class GuardedCalculator {

    // Performs the calculation
    static double calculate(double a, double b, char operator)
            throws DivideByZeroException {

        switch (operator) {
            case '+':
                return a + b;
```

```
            case '-':
                return a - b;

            case '*':
                return a * b;

            case '/':
                if (b == 0) {
                    throw new DivideByZeroException(
                        "Cannot divide by zero."
                    );
                }
                return a / b;

            default:
                throw new IllegalArgumentException(
                    "Invalid operator. Use +, -, *, or /."
                );
        }
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        boolean success = false;
        int attempt = 0;

        while (!success) {

            attempt++;
            System.out.println("\nAttempt " + attempt);

            try {
                System.out.print("Enter first number: ");
```

```
                double a =
Double.parseDouble(sc.nextLine().trim());

                System.out.print("Enter operator (+, -, *, /):
");
                String operatorInput = sc.nextLine().trim();

                if (operatorInput.length() != 1) {
                    throw new IllegalArgumentException(
                        "Enter exactly one operator."
                    );
                }

                char operator = operatorInput.charAt(0);

                System.out.print("Enter second number: ");
                double b =
Double.parseDouble(sc.nextLine().trim());

                double result = calculate(a, b, operator);

                System.out.println("Result: " + result);
                success = true;

            } catch (NumberFormatException e) {
                System.out.println(
                    "Invalid number. Please enter numeric
values."
                );

            } catch (DivideByZeroException e) {
                System.out.println("Calculation error: " +
e.getMessage());

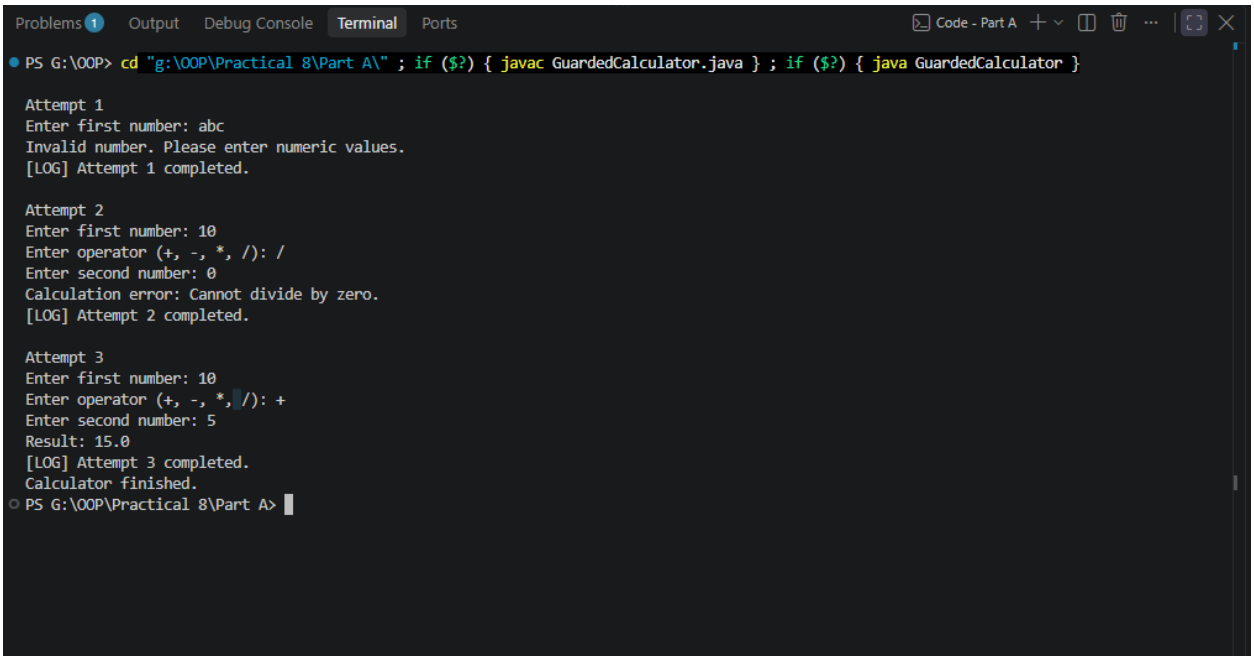
            } catch (IllegalArgumentException e) {
```

```
                System.out.println("Input error: " +
e.getMessage());

            } finally {
                System.out.println(
                    "[LOG] Attempt " + attempt + " completed."
                );
            }
        }

        sc.close();
        System.out.println("Calculator finished.");
    }
}
```

Output:



2. Stock issue: a Warehouse where issue(item, qty) throws a custom checked OutOfStockException carrying the shortfall, and InvalidQuantityException for qty ≤ 0.
Process a list of requests, catching and reporting each failure without stopping the run.

Code:

InvalidQuantityException.java

```
public class InvalidQuantityException extends Exception {

    public InvalidQuantityException(String message) {
        super(message);
    }
}
```

OutOfStockException.java

```
public class OutOfStockException extends Exception {

    private final int shortfall;

    public OutOfStockException(String item, int shortfall) {
        super("Insufficient stock for " + item
                + ". Shortfall: " + shortfall);
        this.shortfall = shortfall;
    }

    public int getShortfall() {
        return shortfall;
    }
}
```

Warehouse.java

```
import java.util.HashMap;
import java.util.Map;

public class Warehouse {

    private final Map<String, Integer> stock = new HashMap<>();

    public Warehouse() {
        stock.put("Laptop", 5);
        stock.put("Mouse", 10);
        stock.put("Keyboard", 3);
    }

    public void issue(String item, int qty)
            throws OutOfStockException,
InvalidQuantityException {

        // Validate quantity first
        if (qty <= 0) {
            throw new InvalidQuantityException(
                "Quantity must be greater than zero."
            );
        }

        // Check whether the item exists
        if (!stock.containsKey(item)) {
            throw new IllegalArgumentException(
                "Unknown item: " + item
            );
        }

        int available = stock.get(item);

        // Reject requests exceeding available stock
```

```
        if (qty > available) {
            int shortfall = qty - available;
            throw new OutOfStockException(item, shortfall);
        }

        // Update stock only after all checks pass
        stock.put(item, available - qty);

        System.out.println(
            "Issued " + qty + " " + item
            + "(s). Remaining stock: " + stock.get(item)
        );
    }

    public void displayStock() {
        System.out.println("\n=== Remaining Stock ===");

        for (Map.Entry<String, Integer> entry :
stock.entrySet()) {
            System.out.println(
                entry.getKey() + ": " + entry.getValue()
            );
        }
    }
}
```

WarehouseDemo.java

```
public class WarehouseDemo {

    // Represents one stock request
    static class Request {
        String item;
        int quantity;

        Request(String item, int quantity) {
            this.item = item;
            this.quantity = quantity;
        }
    }

    public static void main(String[] args) {

        Warehouse warehouse = new Warehouse();

        Request[] requests = {
            new Request("Laptop", 2),
            new Request("Mouse", 4),
            new Request("Laptop", 6),
            new Request("Keyboard", 0),
            new Request("Keyboard", 2),
            new Request("Monitor", 1)
        };

        System.out.println("=== Processing Stock Requests
===");

        for (Request request : requests) {

            System.out.println(
                "\nRequest: " + request.item
                + ", quantity: " + request.quantity
```

```
            );

            try {
                warehouse.issue(request.item,
request.quantity);

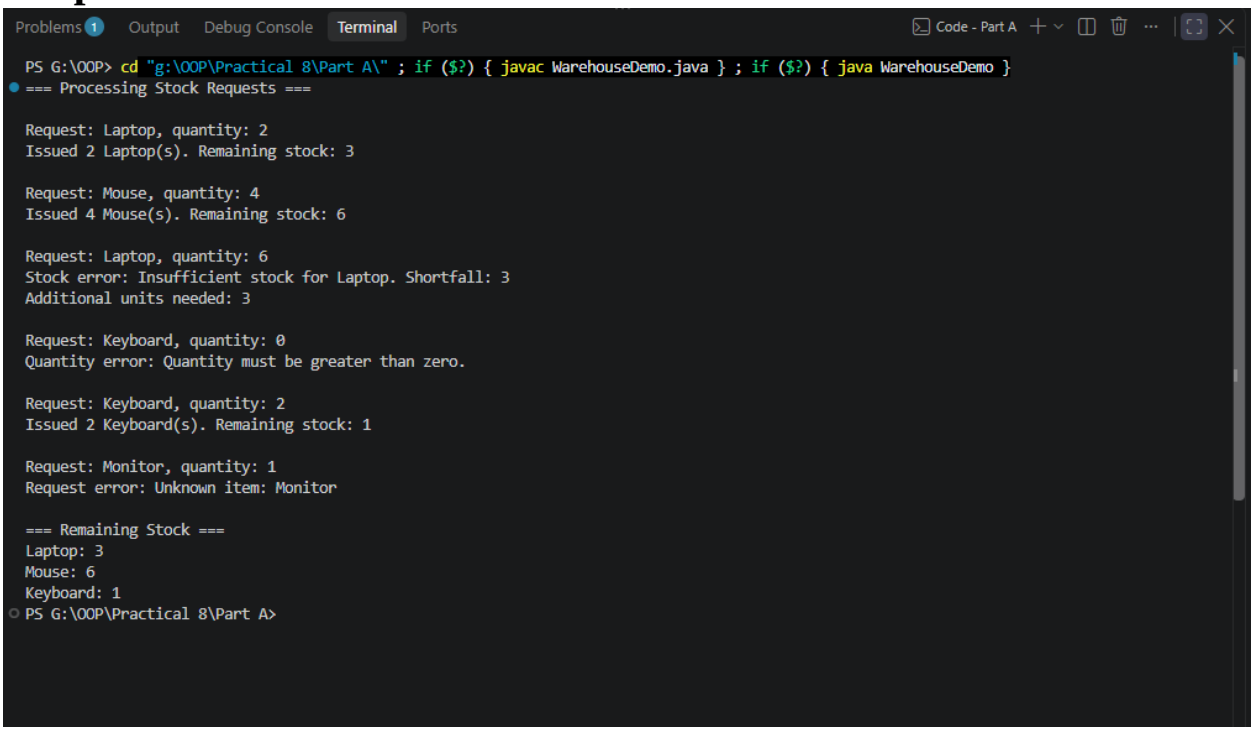
            } catch (OutOfStockException e) {
                System.out.println("Stock error: " +
e.getMessage());
                System.out.println(
                    "Additional units needed: " +
e.getShortfall()
                );

            } catch (InvalidQuantityException e) {
                System.out.println("Quantity error: " +
e.getMessage());

            } catch (IllegalArgumentException e) {
                System.out.println("Request error: " +
e.getMessage());
            }
        }

        warehouse.displayStock();
    }
}
```

Output:


Problems 1
Output
Debug Console
Terminal
Ports
Code - Part A
PS G:\OOP> cd "g:\OOP\Practical 8\Part A\" ; if ($?) { javac WarehouseDemo.java } ; if ($?) { java WarehouseDemo }
=== Processing Stock Requests ===
Request: Laptop, quantity: 2
Issued 2 Laptop(s). Remaining stock: 3
Request: Mouse, quantity: 4
Issued 4 Mouse(s). Remaining stock: 6
Request: Laptop, quantity: 6
Stock error: Insufficient stock for Laptop. Shortfall: 3
Additional units needed: 3
Request: Keyboard, quantity: 0
Quantity error: Quantity must be greater than zero.
Request: Keyboard, quantity: 2
Issued 2 Keyboard(s). Remaining stock: 1
Request: Monitor, quantity: 1
Request error: Unknown item: Monitor
=== Remaining Stock ===
Laptop: 3
Mouse: 6
Keyboard: 1
PS G:\OOP\Practical 8\Part A>

3. (For Advanced Learners – home) Create an AutoCloseable resource and use try-with-resources to prove it always closes — even when an exception is thrown inside the block — and that the original error is the one reported. Decide what to model as the resource.

Code:

ResourceDemo.java

```
import java.io.IOException;

public class ResourceDemo {

    // A resource that must always be closed
    static class FileProcessingSession implements AutoCloseable
{

        public FileProcessingSession() {
            System.out.println("Resource opened.");
        }

        public void process() {
            System.out.println("Processing file...");

            // Simulate an error during processing
            throw new IllegalStateException(
                "Original error: file processing failed."
            );
        }

        @Override
        public void close() throws IOException {
            System.out.println("Resource closed.");

            // Simulate a second error during cleanup
            throw new IOException(
                "Cleanup error: resource release failed."
```

```
            );
        }
    }

    public static void main(String[] args) {

        try (FileProcessingSession session =
                    new FileProcessingSession()) {

            session.process();

        } catch (IllegalStateException e) {

            System.out.println("Caught: " + e.getMessage());

            // Check whether closing also caused an exception
            for (Throwable suppressed : e.getSuppressed()) {
                System.out.println(
                    "Suppressed: " + suppressed.getMessage()
                );
            }
        }
    }
}
```

```
import java.io.IOException;

public class ResourceDemo {

    static class FileProcessingSession implements AutoCloseable
{

        public FileProcessingSession() {
            System.out.println("Resource opened.");
        }

        public void process() {
            System.out.println("Processing file...");

            throw new IllegalStateException(
                "Original error: file processing failed."
            );
        }

        @Override
        public void close() throws IOException {
            System.out.println("Resource closed.");

            throw new IOException(
                "Cleanup error: resource release failed."
            );
        }
    }

    public static void main(String[] args) {

        try (FileProcessingSession session =
                    new FileProcessingSession()) {

            session.process();
```

```
        } catch (IllegalStateException e) {

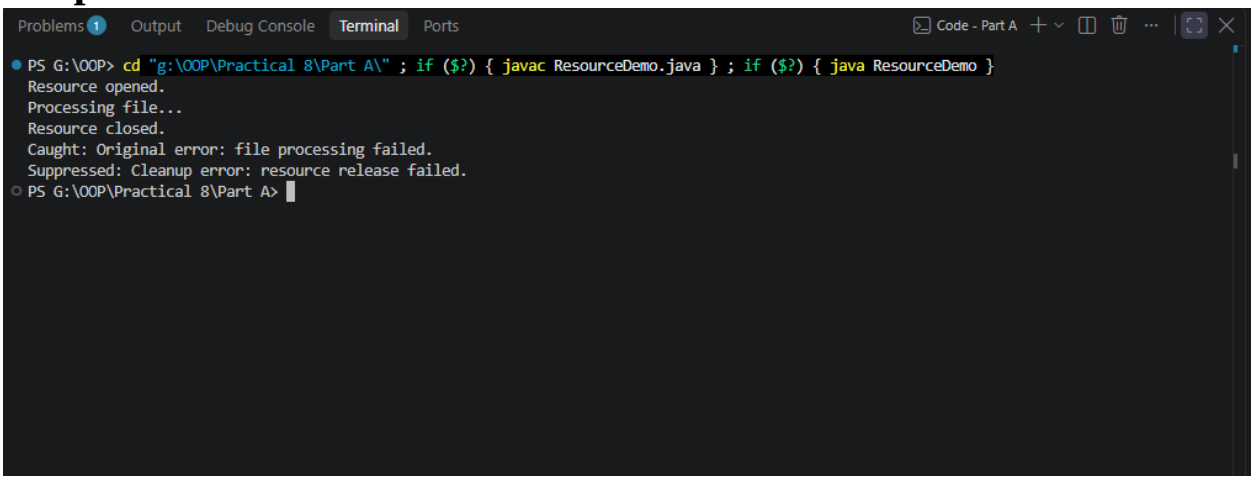
            System.out.println("Caught: " + e.getMessage());

            for (Throwable suppressed : e.getSuppressed()) {
                System.out.println(
                    "Suppressed: " + suppressed.getMessage()
                );
            }

        } catch (IOException e) {

            // Handles an IOException if it is thrown
            // without an earlier exception being propagated.
            System.out.println("Cleanup error: " +
e.getMessage());
        }
    }
}
```

Output:



Key Questions / Analysis / Interpretation to be evaluated during/after Implementation

1. What is the difference between a checked and an unchecked exception?

Ans.

Checked Exception	Unchecked Exception
Checked by the compiler at compile time.	Not required to be handled or declared by the compiler.
Must be caught using try-catch or declared using throws.	Handling or declaring it is optional from the compiler's perspective.
Usually represents conditions an application may anticipate and recover from.	Often represents programming errors or invalid operations.
Examples: IOException, SQLException.	Examples: NullPointerException, ArithmeticException, IllegalArgumentException.

2. What is the purpose of the finally block, and when does it run?

Ans.

The finally block contains code that should execute after a try-catch sequence, whether an exception occurs or not. It is commonly used for cleanup operations and logging.

Example from the Guarded Calculator:

```
try {
    double result = 10 / 2;
    System.out.println(result);

} catch (Exception e) {
    System.out.println("An error occurred.");

} finally {
    System.out.println("Attempt completed.");
}
```

Output:

5.0
Attempt completed.

If an exception occurs, the finally block normally runs after the matching catch block.

3. How does try-with-resources guarantee a resource is closed?

Ans.

Try-with-resources is a Java construct that automatically closes resources implementing AutoCloseable when execution leaves the block.

Example from ResourceDemo.java:

```
try (FileProcessingSession session =
         new FileProcessingSession()) {

    session.process();

}
```

Java automatically calls:
`session.close();`

when the block exits, even if session.process() throws an exception.

The resource's close() method must be called as part of the construct's cleanup process, although the cleanup itself can fail or the JVM can terminate abruptly.